

Contents

1 week 2: data & bias	1
1.1 cleaning a dataset	1
1.1.1 create virtual environment for ml packages 5-10	1
1.1.2 access dataset	2
1.1.3 preliminary exploration 5-10	2
1.1.4 GROUPS: close reading dataset in groups, 20-30 minutes	2
1.1.5 cleaning	3
1.1.6 final step — MultiLabelBinarizer	4
1.1.7 our workflow	5
1.2 break	5
1.3 provocations sharing 30 minutes	5
1.4 continue cleaning	5
1.5 start homework	5

1 week 2: data & bias

1.1 cleaning a dataset

- how to create a virtual environment
- exploring a dataset
- decide what you want to predict
- cleaning the dataset accordingly

1.1.1 create virtual environment for ml packages 5-10

```
# make a new folder for our work
cd Desktop
mkdir 656-notebooks

# create and activate the environment
conda create --name ml-env pandas scikit-learn ipykernel
conda activate ml-env

# now add the kernel to use on Jupyter
ipython kernel install --user --name=ml-env
```

Your conda environment will be installed in
/Users/fcalado/.conda/envs/ml-env/bin/python
Your kernel will be installed at:
/Users/fcalado/Library/Jupyter/kernels/ml-env

1.1.2 access dataset

<https://bit.ly/lgbtq-legislation-2025>

Go to the sheet, navigate to the "Anti-LGBTQ Bills", download as csv.

1.1.3 preliminary exploration 5-10

```
import pandas as pd

df = pd.read_csv("2025-bills.csv")

df.shape
df.info()
df.head()
df.columns
df['State']
df.value_counts('State')
df.value_counts('Bill_Type')
```

1.1.4 GROUPS: close reading dataset in groups, 20-30 minutes

- pass out the printout
- what kind of predictions could we make from the data? 10 min
 - think about which of these columns can be used to guess about another column.
 - * Example:
 - * what states are likely to have "Worst" risk?
- share 20 min
- how to label a bill based on its summary and notes.
 - Summary + Notes = Bill Type

- other things which could be interesting, but we won't pursue (for now):
 - * Sponsors - are there any patterns in who is sponsoring certain bills?
 - * State - are there patterns in which states have certain bill types? or how many bills per state?
- the decisions we make about prediction will affect the way we clean and process the dataset.
- for looking at "worst" we might want to create a new dataframe of just worst bills for Youth State Risk.

```
worst = df['Youth State Risk'] == 'Worst'
```

1.1.5 cleaning

- we are looking to predict bill type based on summary and notes
- we need three columns: summary, notes, bill type
 - spend a couple of minutes looking at the column. Check for inconsistencies, things we'd want to standardize or regularize.
 - what do we need to clean from each column, if at all?

We can also use pandas to do this programmatically. What are the most frequent values for this column?

```
df["Bill Type"].value_counts()
```

There are 298 different labels. That's because we have examples like the one below.

```
"DEI Ban" "DEI Ban " " DEI Ban" "DEI Ban "
```

So we decide that we want to cut down the inconsistent labels. First, preserve the original.

```
df["Bill Type Raw"] = df["Bill Type"]
```

Second, let's split up the long string into individual ones.

```
df["Bill Type"].str.split("/")
```

Third, create a new column for the split labels.

```
df["bill_type_labels"] = df["Bill Type"].str.split("/").fillna("")
```

Fourth, `strip()` to remove white spaces.

```
def clean_labels(labels):  
    return [label.strip() for label in labels]  
  
df["bill_type_labels"] = df["bill_type_labels"].apply(clean_labels)
```

Make the difference between these two columns visible.

```
print(df["Bill Type Raw"].iloc[0])  
print(df["bill_type_labels"].iloc[0])
```

Fifth, pull out the individual label, use `explode()`

```
labels = df["bill_type_labels"].explode()
```

We can ask how many times each label is used!

```
labels.value_counts()  
labels.unique()  
# see the comparison between the original and our new categories.  
len(labels.unique())  
len(df['Bill Type Raw'].unique())
```

Are there any other cleaning steps we'd want to do?

- remove labels that appear 1 time? less than 3 times?

1.1.6 final step — MultiLabelBinarizer

To train a classifier, we would create a "sparse" matrix. A sparse matrix has 0s and 1s to indicate the presence of a category.

We will use the library `sklearn`.

```
from sklearn.preprocessing import MultiLabelBinarizer  
  
mlb = MultiLabelBinarizer()  
  
Y = mlb.fit_transform(df["bill_type_labels"])
```

This creates an `mlb` object and `Y` object. The first contains our classes, and the second is our matrix.

```
# all of our unique labels
mlb.classes_

# scary matrix...
Y

# see that we have the same number of rows as bills
# and we have the same number of columns as labels!!
Y.shape()
```

1.1.7 our workflow

```
RAW BILL TYPE ->
  Preserve original ->
  Split on "/" ->
  Create new column with no NaNs ->
  Clean whitespace ->
  MultiLabelBinarizer ->
  0 / 1 matrix
```

1.2 break

1.3 provocations sharing 30 minutes

- in partners, share your provocations. 5 minutes.
- share out with the class. 10 minutes.

1.4 continue cleaning

1.5 start homework